

2 Numpy Intro (40 points)

All the code/data for this is located in the folder `numpy/`. Each assignment requires you to fill in the blank in a function (in `tests.py` and `warmup.py`) and return the value described in the comment for the function. There's a driver code you do not need to read in `run.py` and `common.py`.

Note: All the `python` below refer to `python3`. As we stated earlier, we are going to use Python 3.7 in this assignment. Python 2 was [sunset](#) on January 1, 2022.

Question 1.1 (the only question): Fill in the code stubs in `tests.py` and `warmups.py`. Put the terminal output in your pdf from:

```
1 python run.py --allwarmups
2 python run.py --alltests
```

Do I have to get every question right? We give partial credit: each warmup exercise is worth 2% of the total grade for this question and each test is worth 3% of the total grade for this question.

2.1 How the Tests Work

When you open one of these two files, you will see starter code that looks like this:

```
1 def sample1(xs):
2     """
3     Inputs:
4     - xs: A list of values
5
6     Returns:
7     The first entry of the list
8     """
9     return None
```

You should fill in the implementation of the function, like this:

```
1 def sample1(xs):
2     """
3     Inputs:
4     - xs: A list of values
5
6     Returns:
7     The first entry of the list
8     """
9     return xs[0]
```

You can test your implementation by running the test script:

```
1 python run.py --test w1          # Check warmup problem w1 from warmups.py
2 python run.py --allwarmups       # Check all the warmup problems
3 python run.py --test t1         # Check the test problem t1 from tests.py
4 python run.py --alltests        # Check all the test problems
5
6 # Check all the warmup problems; if any of them fail, then launch the pdb
7 # debugger so you can find the difference
8 python run.py --allwarmups --pdb
```

If you are checking all the warmup problems (or test problems), the perfect result will be:

```

1 python run.py --allwarmups
2 Running w1
3 Running w2
4 ...
5 Running w20
6 Ran warmup tests
7 20/20 = 100.0

```

2.2 Warmup Problems

You need to solve all 20 of the warmup problems in `warmups.py`. They are all solvable with one line of code.

2.3 Test Problems

You need to solve all 20 problems in `tests.py`. Many are not solvable in one line. You may not use a loop to solve any of the problems, although you may want to first figure out a slow for-loop solution to make sure you know what the right computation is, before changing the for-loop solution to a non for-loop solution. The one exception to the no-loop rule is `t10` (although this can also be solved without loops).

Here is one example:

```

1 def t4(R, X):
2     """
3     Inputs:
4     - R: A numpy array of shape (3, 3) giving a rotation matrix
5     - X: A numpy array of shape (N, 3) giving a set of 3-dimensional vectors
6
7     Returns:
8     A numpy array Y of shape (N, 3) where Y[i] is X[i] rotated by R
9
10    Par: 3 lines
11    Instructor: 1 line
12
13    Hint:
14    1) If v is a vector, then the matrix-vector product Rv rotates the vector
15       by the matrix R.
16    2) .T gives the transpose of a matrix
17    """
18    return None

```

2.4 What We Provide

For each problem, we provide:

Inputs: The arguments that are provided to the function

Returns: What you are supposed to return from the function

Par: How many lines of code it should take. We don't grade on this, but if it takes more lines than this, there is probably a better way to solve it. **Except for `t10`, you should not use any explicit loops.**

Instructor: How many lines our solution takes. Can you do better?

Hints: Functions and other tips you might find useful for this problem.

2.5 Walkthroughs and Hints

Test 8: If you get the axes wrong, numpy will do its best to make sure that the computations go through but the answer will be wrong. If your mean variable is 1×1 , you may find yourself with a matrix where the full matrix has mean zero. If your standard deviation variable is $1 \times M$, you may find each column has standard deviation one.

Test 9: This sort of functional form appears throughout data-processing. This is primarily an exercise in writing out code with multiple nested levels of calculations. Write each part of the expression one line at a time, checking the sizes at each step.

Test 10: This is an exercise in handling weird data formats. You may want to do this with for loops first.

1. First, make a loop that calculates the vector $C[i]$ that is the centroid of the data in $X_S[i]$. To figure out the meaning of things, there is no shame in trying operations and seeing which produce the right shape. Here we have to specify that the centroid is M -dimensions to point out how we want $X_S[i]$ interpreted. The centroid (or average of the vectors) has to be calculated with the average going down the columns (i.e., rows are individual vectors and columns are dimensions).
2. Allocate a matrix that can store all the pairwise distances. Then, double for loop over the centroids i and j and produce the distances.

Test 11: You are given a set of vectors $\mathbf{x}_1, \dots, \mathbf{x}_N$ where each vector $x_i \in \mathbb{R}^M$. These vectors are stacked together to create a matrix $\mathbf{X} \in \mathbb{R}^{N \times M}$. Your goal is to create a matrix $\mathbf{D} \in \mathbb{R}^{N \times N}$ such that $\mathbf{D}_{i,j} = \|\mathbf{x}_i - \mathbf{x}_j\|$. Note that $\|\mathbf{x}_i - \mathbf{x}_j\|$ is the L2-norm or the Euclidean length of the vector. The useful identity you are given is that $\|\mathbf{x} - \mathbf{y}\|^2 = \|\mathbf{x}\|^2 + \|\mathbf{y}\|^2 - 2\mathbf{x}^T \mathbf{y}$. This is true for any pair of vectors $\mathbf{x}$ and $\mathbf{y}$ and can be used to calculate the distance quickly.

At each step, your goal is to replace slow but correct code with fast and correct code. If the code breaks at a particular step, you know where the bug is.

1. First, write a correct but slow solution that uses two for loops. In the inner body, you should plug in the given identity to make $\mathbf{D}_{i,j} = \|\mathbf{x}_i\|^2 + \|\mathbf{x}_j\|^2 - 2\mathbf{x}_i^T \mathbf{x}_j$. Do this in three separate terms.
2. Next, write a version that first computes a matrix that contains all the dot products, or $\mathbf{P} \in \mathbb{R}^{N \times N}$ such that $\mathbf{P}_{i,j} = \mathbf{x}_i^T \mathbf{x}_j$. This can be done in a single matrix-matrix operation. You can then calculate the distance by doing $\mathbf{D}_{i,j} = \|\mathbf{x}_i\|^2 + \|\mathbf{x}_j\|^2 - 2\mathbf{P}_{i,j}$.
3. Finally, calculate a matrix containing the norms, or a $\mathbf{N} \in \mathbb{R}^{N \times N}$ such that $\mathbf{N}_{i,j} = \|\mathbf{x}_i\|^2 + \|\mathbf{x}_j\|^2$. You can do this in two stages: first, calculate the squared norm of each row of $\mathbf{X}$ by summing the squared values across the row. Suppose $\mathbf{S}$ is this array (i.e., $\mathbf{S}_i = \|\mathbf{x}_i\|^2$ and $\mathbf{S}_i \in \mathbb{R}^N$), but be careful that you look at the *shape* that you get as output. If you compute $\mathbf{S} + \mathbf{S}^T$, you should get $\mathbf{N}$. Now you can calculate the distance inside the double for loop as $\mathbf{D}_{i,j} = \mathbf{N}_{i,j} - 2\mathbf{P}_{i,j}$.
4. The double for loop is now not very useful. You can just add/scale the two arrays together element-wise.

Test 18: Here you draw a circle by finding all entries in a matrix that are within r cells of a row y and column x . This is a not particularly intellectually stimulating exercise, but it is practice in writing (and debugging) code that reasons about rows and columns.

1. First, write a correct but slow solution that uses two for loops. In the inner body, plug in the correct test for the given `i, j`. Make sure the test passes; be careful about rows and columns.
2. Look at the documentation for `np.meshgrid` briefly. Then call it with `np.arange(3)` and `np.arange(5)` as arguments. See if you can create two arrays such that `IndexI[i, j] = i` and `IndexJ[i, j] = j`.
3. Replace your test that uses two for loops with something that just uses `IndexI` and `IndexJ`.

3 Data Interpretation and Making Your Own Visualization (10 points total)

Throughout the course, a lot of the data you have access to will be in the form of an image. These won't be stored and saved in the same format that you're used to when interacting with ordinary images, such as off your cell phone: sometimes they'll have negative values, really really big values, or invalid values. If you can look at images quickly, then you'll find bugs quicker. If you **only** print debug, you'll have a bad time. To teach you about interpreting things, I've got a bunch of mystery data¹ that we'll analyze together. You'll write a brief version of the important `imsave` function for visualizing.

Let's load some of this mysterious data.

```
1 >>> X = np.load("mysterydata/mysterydata.npy")
2 >>> X
3 array([[0, 0, 0, ..., 0, 0, 1],
4        [0, 0, 0, ..., 0, 0, 0],
5        [0, 0, 0, ..., 0, 0, 0],
6        ...,
7        [0, 0, 0, ..., 0, 0, 0],
8        [0, 0, 0, ..., 0, 0, 0],
9        [0, 0, 0, ..., 0, 0, 0]],
10
11        ... (some more zeros) ...
12
13        [0, 0, 0, ..., 0, 0, 0],
14        [0, 0, 0, ..., 0, 0, 0],
15        [0, 0, 0, ..., 0, 0, 0],
16        ...,
17        [0, 0, 0, ..., 0, 0, 0],
18        [0, 0, 0, ..., 0, 0, 0],
19        [0, 0, 0, ..., 0, 0, 0]]], dtype=uint32)
```

Looks like it's a bunch of zeros. Nothing to see here folks! For better or worse: Python only shows the sides of an array when printing it and the sides of images that have gone through some processing tend to not be representative.

After you print it out, you should always look at the **shape** of the array and **the data type**.

```
1 >>> X.shape
2 (512, 512, 9)
3 >>> X.dtype
4 dtype('uint32')
```

The shape and datatype are really important. If something is an unexpected shape, that's **really bad**. This is similar to doing a physics problem where you're trying to measure the speed of light (which should be in m/s), and getting a result that is in gauss. The computer may happily chug along and produce a number, but if the units or shape are wrong, the answer is almost certainly wrong. The datatype is also really important, because data type conversion can be lossy: if you have values between 0 and 1, and you convert to an integer format, you'll end up with 0s and 1s.

In this particular case, the data has height 512 (the first dimension), width 512 (the second), and 9 channels (the last). These order of dimensions here is a *convention*, meaning that there are multiple equivalent options. For instance, in other conventions, the channel is the first dimension. Generally, you'll be given data that is

¹For the curious, this is data that originates from [Solar Dynamics Observatory](#), and in particular the AIA instrument. These are, to a first approximation, photos of the sun in wavelengths that in the ultraviolet and extreme UV, and which primarily form in the upper part of the Sun's atmosphere.

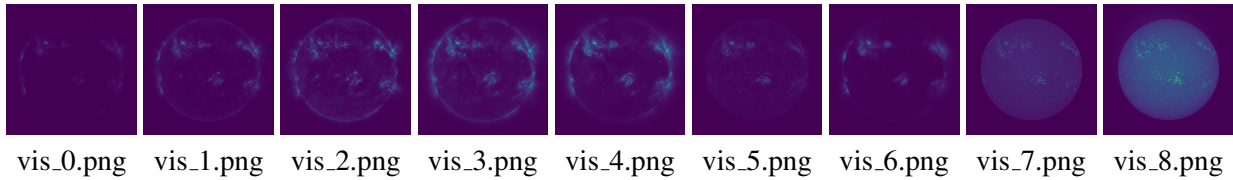


Figure 1: The Mystery Data

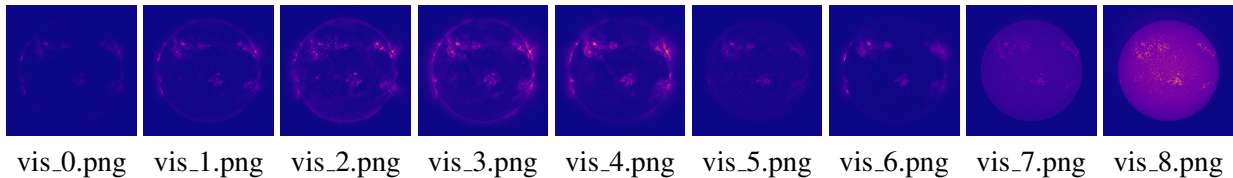


Figure 2: The Mystery Data, Visualized with the Plasma Colormap

HxWxC and we'll try to be clear about how data is stored. If later on, you're given some other piece of data, figuring out the convention is a bit like rotating a picture until it looks right: figure out what you expect, and flip things until it looks like you'd expect.

If you've got an image, after you print it, you probably want to *visualize* the output.

We don't see in 9 color channels, and so you can't look at them all at once. If you've got a bug, one of the most important things to do is to look at the image. You can look at either the first channel or all of the channels as individual images

```
1 >>> plt.imsave("vis.png",X[:, :, 0])
```

You can see what the output looks like in Figure 1. This is a false color image. You can read about these in Section A.3, but the short version is that the given image, you find the minimum value (`vmin`) and the maximum value (`vmax`) and assign colors based on where each pixel's value falls between those. These colors look like: Low  High. `plt.imsave` finds `vmin` and `vmax` for you on its own by computing the minimum and maximum of the array.

If you'd like to look at all 9 channels, save 9 images:

```
1 >>> for i in range(9):
2 ...     plt.imsave("vis_%d.png" % i,X[:, :, i])
3 ...
```

If you're inside a normal python interpreter, you can do a for loop. If you're inside a debugger, it's sometimes hard to do a for loop. If you promise not to tell your programming languages friends, you can use side effects to do a for loop to save the data.

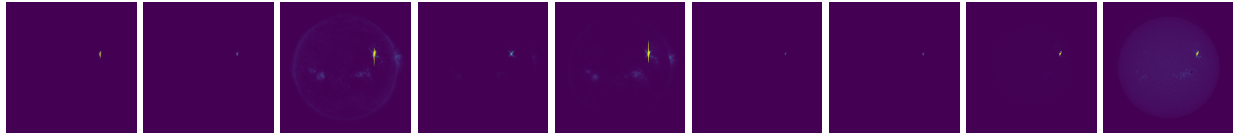
```
1 (Pdb) [plt.imsave("vis_%d.png" % i,X[:, :, i]) for i in range(9)]
2 [None, None, None, None, None, None, None, None, None]
```

The list of Nones is just the return value of `plt.imsave`, which saves stuff to disk and returns a `None`.

If you'd like to change the colormap, you can specify this with `cmap`. For instance

```
1 >>> for i in range(9):
2 ...     plt.imsave("vis_%d.png" % i,X[:, :, i],cmap='plasma')
```

produces the outputs in Figure 2. These use the plasma colormap which looks like: Low  High.



vis2_0.png vis2_1.png vis2_2.png vis2_3.png vis2_4.png vis2_5.png vis2_6.png vis2_7.png vis2_8.png

Figure 3: The Mystery Data #2.

3.1 Images with wide ranges of values

Report 3.1 (pictures, 2 points) Try loading `mysterydata2.npy` and visualizing it. You should get something like Figures 3. It's hard to see stuff because one spot is *really* bright. In this case, it's because there's a solar flare that's producing immense amounts of light. A common trick for making things easier to see is to apply a nonlinear correction. Here are a few options:

$$p^\gamma \text{ with } \gamma \in [0, 1] \quad \text{or} \quad \log(p) \quad \text{or} \quad \log(1 + p) \quad (1)$$

where the last one can be done with `np.log1p`. Apply a nonlinear correction to the second mystery data and visualize it. **Put two of the channels (i.e., `X[:, :, i]` as images into the report. You can stick them side-by-side.**

3.2 Images that Screw up and Knowing Your Limits

vis3_0.png vis3_1.png vis3_2.png vis3_3.png vis3_4.png vis3_5.png vis3_6.png vis3_7.png vis3_8.png

Figure 4: The Mystery Data # 3, Visualized!

Let's try this again, using one of the other data.

```
1 >>> X = np.load("mysterydata/mysterydata3.npy")
2 >>> for i in range(9):
3 ...     plt.imsave("vis3_%d.png" % i, X[:, :, i])
4 ...
```

The results are shown in Figure 4. They're all white. What's going on?

If you've got an uncooperative piece of data that won't visualize or produces buggy results, it's worth checking to see if all the values are reasonable. One option that'll cover all your cases is `np.isfinite`, which is `False` for values that are NaN (not a number) or $\pm\infty$ and `True` otherwise. If you then take the mean over the array, you get the fraction of entries that are normal-ish. If the mean value is *anything* other than 1, you may be in trouble. Here:

```
1 >>> np.mean(np.isfinite(X))
2 0.6824828253851997
```

Alternatively, this also works

```
1 >>> np.sum(~np.isfinite(X))
2 749117
```

Other things that check things are `np.isnan` (which returns True for NaNs) and `np.isinf` (which returns True for infinite values). Even a single NaN in your data is a problem: any number that touches another NaN turns into a NaN. The totally white values happen because `plt.imshow` tries to find `vmin/vmax`, and the minimum of a value and a NaN is a NaN. The resulting color is a NaN as well. If you've got NaNs in your data, many functions you may want to use (e.g., `mean`, `sum`, `min`, `max`, `median`, `quantile`) have a `nan` version.

Report 3.2 (some pictures, 2 points) Fix the images by determining the right range (use `np.nanmin`, `np.nanmax`) and then pass arguments into `plt.imshow` to set the range for the visualization. To figure out what arguments to set, look at the documentation for `plt.imshow`. Put two images from `mysterydata` 3 in the report.

3.3 Rolling Your Own `plt.imshow`

You'll make your own `plt.imshow` by filling in `colormapArray`. Here's how the false color image works: You're given a $H \times W$ matrix X and a colormap matrix C that is $N \times 3$ where each row is a color red/green/blue. You produce a $H \times W \times 3$ matrix O . Each scalar-valued pixel $X[i, j]$ gets converted into red/green/blue values for $O[i, j, :]$ following this rule: if the pixel has value v the corresponding output color comes from a row determined by (approximately)

$$(N - 1) \frac{(v - v_{\min})}{(v_{\max} - v_{\min})}. \quad (2)$$

However, you'll have to be very careful – this precise equation won't always work. As an exercise – can you spot something that might cause the expression to be a NaN?

Coding 3.3 (3 points): Fill in `colormapArray`. To test, you'll have to write some calling code in the main part. You can use either `plt.imshow` or `cv2.imwrite` to save the image to an file.

Report 3.4 (3 points): Visualize `mysterydata4.npy` using your system without it crashing and put all nine images into your report. You may have to make a design decision about what to do about results that are undefined. If the results are undefined, then any option that seems reasonable is fine. Your colormap should look similar to Figure 2. If the colors look inverted, see Beware 3!

Beware 1! There are a bunch of edge cases in the equation for the color: it won't always return an integer between 0 and $N - 1$. It will also definitely blow up under certain input conditions (also, watch the type).

Beware 2! You're asked by the code to return a $H \times W$ `uint8` image. There are a lot of shortcuts/implied sizes and shapes in computer vision notation – since this is a $H \times W$ *color* image, it should have 3 channels (i.e., be $H \times W \times 3$). Since it's `uint8`, you should make the image go from 0 to 255 before returning it (otherwise everything gets clipped to 0 and 1, which correspond to the two lowest brightness settings). Like all other jargon, this is annoying until it is learned; after it is learned, it is then useful.

Beware 3! If you choose to save the results using `opencv`, you may have blue and red flipped – `opencv` assumes blue is first and the rest of the world assumes red is first. You can identify this by the fact that the columns of the are defined as Red/Green/Blue and there is a lot of blue and not much red in the lowest entry.